

02b Tree-Based Time Series Models (Random Forest and XGBoost)

Objective: build leakage-safe tree-based forecasts for monthly credit reporting share, using the same split/evaluation framing as Notebook 02a for direct comparability.

This notebook uses engineered monthly features and evaluates:

- Random Forest multi-output direct forecasting ($h=1..4$)
- XGBoost direct per-horizon forecasting ($h=1..4$, one model per horizon)
- compact hyperparameter grid search for both model families

Horizon scope note:

- `FORECAST_HORIZON = 12` is retained for alignment with 02a framing,
- tree experiments in this notebook are implemented and evaluated on short-lead direct horizons ($h=1..4$) due sample-size and feature-window constraints.

```
In [4]: # =====
# IMPORT STATEMENTS
# =====
# Core libraries for data prep, feature engineering, modeling, and diagnostics.

import os
import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Compatibility guard for environments with optional pyarrow/sklearn binary mismatch
os.environ.setdefault("SKLEARN_ALLOW_INVALID_PYARROW", "1")

SKLEARN_AVAILABLE = True
try:
    from sklearn.preprocessing import StandardScaler
    from sklearn.ensemble import RandomForestRegressor
    from sklearn.multioutput import MultiOutputRegressor
except Exception as e:
    SKLEARN_AVAILABLE = False
    print("scikit-learn import unavailable in this kernel:", e)

# XGBoost imported for upcoming section extension.
import xgboost as xgb

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

```
warnings.filterwarnings("ignore")
```

```
In [5]: # =====
# DATA SETUP + SHARED HELPERS (MATCH 03a SPLIT/EVAL STYLE)
# =====
# Load credit reporting target series and enforce same split logic.

FREQ_ALIAS = "M"
SEASONAL_PERIODS = 12
FORECAST_HORIZON = 12
OUTPUT_TAG = "02b_creditreporting_monthly"

data_path = r"..\\data\\processed\\credit_reporting_timeseries_all.parquet"
df = pd.read_parquet(data_path)

required_cols = {
    "year_month",
    "credit_reporting_count",
    "total_complaints",
    "consumer_consent_yes_count",
    "timely_response_yes_count",
    "consumer_disputed_yes_count",
    "credit_reporting_share",
}
missing = required_cols.difference(df.columns)
if missing:
    raise ValueError(
        f"Missing required columns in credit_reporting_timeseries_all.parquet: {sorted(missing)}"
    )

# Build period-indexed monthly frame using the provided schema only.
ts_df = df[list(required_cols)].copy()
ts_df["period"] = pd.to_datetime(ts_df["year_month"], errors="coerce").dt.to_period("M")
ts_df = ts_df.dropna(subset=["period", "credit_reporting_share"]).sort_values("period")
ts_df = ts_df.groupby("period", as_index=False).first()

# Remove leading rows where share is zero (e.g., early history before product appearance)
non_zero_share = ts_df["credit_reporting_share"].astype(float) > 0
if non_zero_share.any():
    first_non_zero_loc = int(non_zero_share.values.argmax())
    ts_df = ts_df.iloc[first_non_zero_loc:].copy()

# Target for tree-based modeling.
series_df = ts_df[["period", "credit_reporting_share"]].rename(columns={"credit_reporting_share": "y"})
y = series_df["y"].astype(float)

# Use all loaded parquet columns except the requested exclusions and target/date features
excluded_feature_cols = {
    "credit_reporting_count",
    "total_complaints",
    "credit_reporting_share",
    "period",
    "year_month",
}
tree_feature_df = ts_df.drop(columns=[c for c in excluded_feature_cols if c in ts_df.columns])
```

```

tree_feature_df.index = series_df.index

n = len(y)
test_size = max(4, n // 5)
val_size = test_size
while n - (val_size + test_size) < 8 and test_size > 2:
    test_size -= 1
    val_size = test_size

train = y.iloc[: n - val_size - test_size].copy()
val = y.iloc[n - val_size - test_size : n - test_size].copy()
test = y.iloc[n - test_size :].copy()

print(f"Source: {data_path}")
print(f"Rows after removing leading zero-share periods: {len(y)}")
print(f"Tree feature columns used: {tree_feature_df.shape[1]}")
print(f"Frequency: {FREQ_ALIAS} | Train={len(train)}, Val={len(val)}, Test={len(test)}")
display(series_df.head())

```

Source: ..\data\processed\credit_reporting_timeseries_all.parquet
Rows after removing leading zero-share periods: 159
Tree feature columns used: 3
Frequency: M | Train=97, Val=31, Test=31

y	
period	
2012-10	0.053405
2012-11	0.127382
2012-12	0.117185
2013-01	0.093522
2013-02	0.127560

```

In [6]: # Shared metrics + result/plot helpers for standalone execution
def smape(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    denom = np.abs(y_true) + np.abs(y_pred)
    ratio = np.where(denom == 0, 0.0, 2.0 * np.abs(y_true - y_pred) / denom)
    return float(100.0 * np.mean(ratio))

def mae(y_true, y_pred):
    return float(np.mean(np.abs(np.asarray(y_true) - np.asarray(y_pred))))

def rmse(y_true, y_pred):
    return float(np.sqrt(np.mean((np.asarray(y_true) - np.asarray(y_pred)) ** 2)))

eval_rows = []

def append_result(model_name, val_pred, test_pred, runtime_sec):
    vp = np.asarray(val_pred, dtype=float)
    tp = np.asarray(test_pred, dtype=float)

```

```

yt_v = y_val_multi.iloc[:, 0].values
yt_t = y_test_multi.iloc[:, 0].values
eval_rows.append({
    "Model": model_name,
    "sMAPE_val": smape(yt_v, vp[:, 0] if vp.ndim > 1 else vp),
    "sMAPE_test": smape(yt_t, tp[:, 0] if tp.ndim > 1 else tp),
    "MAE_val": mae(yt_v, vp[:, 0] if vp.ndim > 1 else vp),
    "MAE_test": mae(yt_t, tp[:, 0] if tp.ndim > 1 else tp),
    "RMSE_val": rmse(yt_v, vp[:, 0] if vp.ndim > 1 else vp),
    "RMSE_test": rmse(yt_t, tp[:, 0] if tp.ndim > 1 else tp),
    "runtime_sec": float(runtime_sec),
})

def plot_section_predictions(title, pred_map):
    fig, axes = plt.subplots(1, 2, figsize=(14, 4), sharey=True)
    for name, (vp, tp) in pred_map.items():
        v = vp[:, 0] if np.asarray(vp).ndim > 1 else np.asarray(vp)
        t = tp[:, 0] if np.asarray(tp).ndim > 1 else np.asarray(tp)
        axes[0].plot(y_val_multi.index.astype(str), y_val_multi.iloc[:, 0].values,
                    axes[0].plot(y_val_multi.index.astype(str), v, label=name)
        axes[1].plot(y_test_multi.index.astype(str), y_test_multi.iloc[:, 0].values,
                    axes[1].plot(y_test_multi.index.astype(str), t, label=name)
    axes[0].set_title(f"{title} - Validation")
    axes[1].set_title(f"{title} - Test")
    for ax in axes:
        ax.tick_params(axis="x", rotation=45)
        ax.grid(True, alpha=0.3)
        ax.legend()
    plt.tight_layout()
    plt.show()

```

1. Feature Engineering + Leakage-Safe Pipeline

We use direct multi-horizon forecasting with strict temporal guardrails.

Definitions:

- direct multi-horizon: separate target columns for `t+1` to `t+4` (`y_h1` to `y_h4`)
- leakage: any feature using information not available at prediction time
- exogenous features: non-target variables derived from monthly aggregates (for example consent/timely/disputed counts)

Horizon handling:

- model training/evaluation in this notebook is explicitly on `h=1..4`
- this short-lead horizon is deliberate for tree models under current sample size
- 02a remains the full 12-month baseline reference, and 02b comparisons are interpreted in that context

Leakage controls implemented:

- lag/rolling/EWMA features use past and current information only

- horizon targets are created by forward shifts of the target
- scaling is fit on train only, then applied to validation/test
- split order is strictly time-based

Implementation Specification (Reviewer Quick Reference)

This table consolidates the concrete implementation choices used in this notebook.

Component	Multi-step strategy	Horizon handling	Parameter choices in this notebook
Feature-target construction	Direct multi-horizon target matrix	Explicit short-lead setup: <code>y_h1..y_h4</code>	Targets built by forward shifts; leakage-safe lag/rolling/EWMA + retained monthly covariates
Random Forest (base run)	Direct multi-output (<code>MultiOutputRegressor</code>)	Trained/evaluated on <code>h=1..4</code> ; macro and per-horizon summaries reported	Base config used in Section 2: <code>n_estimators=500</code> , <code>max_depth=6</code> , <code>min_samples_leaf=2</code> , <code>random_state=42</code>
XGBoost (base run)	Direct per-horizon (one model per horizon, stacked output)	Trained/evaluated on <code>h=1..4</code>	Base config in Section 3 includes <code>eta=0.1</code> , <code>max_depth=6</code> , <code>objective='reg:squarederror'</code> , <code>subsample=0.9</code> , <code>colsample_bytree=0.9</code> , <code>num_boost_round=200</code>
RF grid search	Direct multi-output	Same <code>h=1..4</code> target design	Search grid: <code>n_estimators=[300,500]</code> , <code>max_depth=[4,6,8]</code> , <code>min_samples_leaf=[1,2]</code>
XGB grid search	Direct per-horizon	Same <code>h=1..4</code> target design	Search grid: <code>num_boost_round=[100,200,300]</code> , <code>eta=[0.05,0.1]</code> , <code>max_depth=[4,6]</code> (other params fixed)

Global evaluation protocol:

- `FORECAST_HORIZON=12` is retained for framing alignment with 02a, but tree model training/evaluation here is explicitly short-lead `h=1..4`,
- comparisons use the same metric family (sMAPE/MAE/RMSE + runtime) and are interpreted with validation-size sensitivity, significance, horizon-step, and bias diagnostics.

```
In [7]: # =====
# SECTION 1: FEATURE ENGINEERING + LEAKAGE-PROOF PREP
# =====
# Direct multi-horizon targets and leakage-safe feature matrix construction.

# ---- Multi-horizon direct targets (h=1..4) ----
y1 = y.shift(-1)
y2 = y.shift(-2)
y3 = y.shift(-3)
y4 = y.shift(-4)
y_multi = pd.concat([y1, y2, y3, y4], axis=1)
y_multi.columns = ["y_h1", "y_h2", "y_h3", "y_h4"]

# ---- Feature block on full y (no future leak) ----
feat = pd.DataFrame(index=y.index)

# Lags: y(t-1) ... y(t-12)
for l in range(1, 13):
    feat[f"lag_{l}"] = y.shift(l)

# Rolling windows: local and annual-like
feat["roll_mean_4"] = y.rolling(4).mean()
feat["roll_std_4"] = y.rolling(4).std()
feat["roll_mean_12"] = y.rolling(12).mean()
feat["roll_std_12"] = y.rolling(12).std()

# Seasonal rolling proxy
feat["seasonal_roll_mean_4"] = y.rolling(4).mean()
feat["seasonal_roll_std_4"] = y.rolling(4).std()

# EWMA with alpha=0.3
feat["ewm_a03"] = y.ewm(alpha=0.3, adjust=False).mean()

# Calendar / trend features
feat["cum_q"] = np.arange(len(y), dtype=float)
feat["year_since_2012"] = (y.index.year - 2012).astype(float)
feat["days_since_start"] = (y.index.to_timestamp(how="end") - y.index[0].to_timestamp(how="start")).astype(float)

# Month dummies (for monthly frequency)
m_dummies = pd.get_dummies(y.index.month, prefix="M", drop_first=False).astype(int)
m_dummies.index = y.index
feat = pd.concat([feat, m_dummies], axis=1)

# Fourier terms for monthly seasonality, k=1,2
for k in [1, 2]:
    feat[f"sin_k{k}"] = np.sin(2 * np.pi * k * feat["cum_q"] / 12.0)
    feat[f"cos_k{k}"] = np.cos(2 * np.pi * k * feat["cum_q"] / 12.0)
```

```

# Linear trend residual feature
coef = np.polyfit(feats["cum_q"].values, y.values, 1)
trend_line = coef[0] * feats["cum_q"].values + coef[1]
feats["trend_linear"] = trend_line
feats["trend_resid"] = y.values - trend_line

# Add all retained parquet features (except excluded cols handled in setup cell).
if "tree_feature_df" in globals() and tree_feature_df is not None and not tree_feat
    exog = tree_feature_df.reindex(y.index).copy()
    exog = pd.get_dummies(exog, drop_first=False, dtype=int)
    exog = exog.apply(pd.to_numeric, errors="coerce")
    feats = pd.concat([feats, exog], axis=1)

# Align X and y_multi, dropping rows with any NA from lag/rolling/target shifts
frame = pd.concat([feats, y_multi], axis=1).dropna().copy()

X = frame[feats.columns]
Y = frame[y_multi.columns]

# Required split alignment using train length from setup split
split_train_end = len(train) - 4
split_val_end = len(train)

X_train = X.iloc[:split_train_end].copy()
X_val = X.iloc[split_train_end:split_val_end].copy()
X_test = X.iloc[split_val_end:].copy()

y_train_multi = Y.iloc[:split_train_end].copy()
y_val_multi = Y.iloc[split_train_end:split_val_end].copy()
y_test_multi = Y.iloc[split_val_end:].copy()

# Scale on train only to avoid leakage; trees are scale-invariant, but kept for con
sk_ok = globals().get("SKLEARN_AVAILABLE", False)
if sk_ok:
    scaler = StandardScaler()
    X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), index=X_train.inde
    X_val_scaled = pd.DataFrame(scaler.transform(X_val), index=X_val.index, columns
    X_test_scaled = pd.DataFrame(scaler.transform(X_test), index=X_test.index, colu
else:
    X_train_scaled = X_train.copy()
    X_val_scaled = X_val.copy()
    X_test_scaled = X_test.copy()

print("Feature matrix shape:", X.shape)
print("Target matrix shape:", Y.shape)
print("Split shapes:")
print(" X_train:", X_train_scaled.shape, "| y_train_multi:", y_train_multi.shape)
print(" X_val:", X_val_scaled.shape, "| y_val_multi:", y_val_multi.shape)
print(" X_test:", X_test_scaled.shape, "| y_test_multi:", y_test_multi.shape)

# ACF/PACF diagnostics up to lag 24 for monthly structure check
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(y.values, lags=24, ax=axes[0])
axes[0].set_title("ACF (lags 1-24)")

```

```
plot_pacf(y.values, lags=24, ax=axes[1], method="ywmm")
axes[1].set_title("PACF (lags 1-24)")
plt.tight_layout()
plt.show()
```

Feature matrix shape: (143, 43)

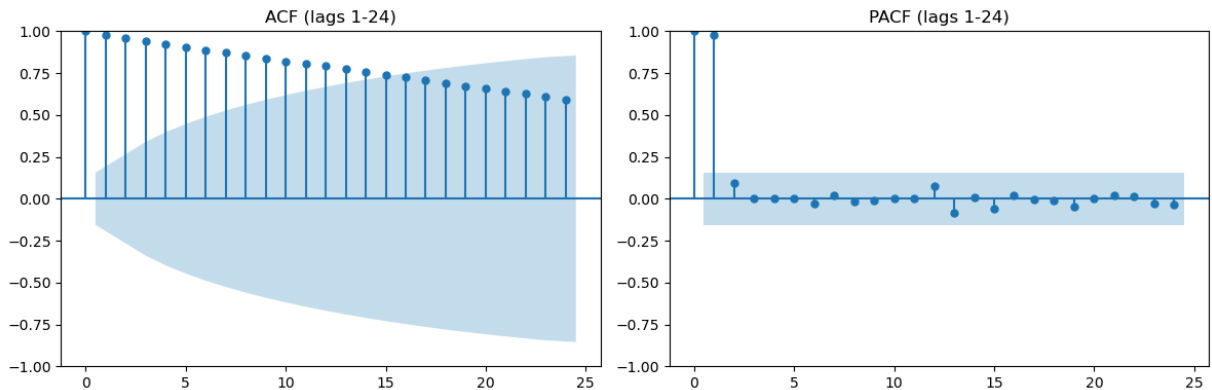
Target matrix shape: (143, 4)

Split shapes:

X_train: (93, 43) | y_train_multi: (93, 4)

X_val: (4, 43) | y_val_multi: (4, 4)

X_test: (46, 43) | y_test_multi: (46, 4)



2. Random Forest (Multi-Horizon Direct)

We fit `MultiOutputRegressor(RandomForestRegressor)` to predict `y_h1` to `y_h4` simultaneously.

Forecast-design clarification:

- this is a **direct multi-horizon** setup, not recursive forecasting,
- `MultiOutputRegressor` trains one separate RF estimator per target column (`y_h1` , `y_h2` , `y_h3` , `y_h4`) using the same feature matrix,
- horizon `h+2` to `h+4` predictions do **not** consume earlier predicted values as inputs.

Evaluation outputs include:

- macro summary metrics (validation and test)
- per-horizon metric table (`h=1` to `h=4`)
- horizon-1 trajectory plot (actual vs predicted)
- feature-importance chart from the `h=1` estimator

Current run context:

- RF performs strongly on validation but underpredicts the higher test-period level, which increases test error relative to statistical baselines.

```
In [10]: # =====
# SECTION 2: RANDOM FOREST (MULTI-OUTPUT, DIRECT HORIZONS)
# =====
```

```

t0 = time.perf_counter()
rf = MultiOutputRegressor(
    RandomForestRegressor(
        n_estimators=500,
        max_depth=6,
        random_state=42,
        n_jobs=-1
    )
)
rf.fit(X_train_scaled, y_train_multi)
runtime = time.perf_counter() - t0

val_pred = rf.predict(X_val_scaled)
test_pred = rf.predict(X_test_scaled)

append_result("RF-Multi", val_pred, test_pred, runtime)
plot_section_predictions("RF Multi-Horizon (h=1 shown)", {"RF": (val_pred, test_pre

# Per-horizon sMAPE/MAE/RMSE table
h_rows = []
for h in range(4):
    h_rows.append({
        "Horizon": f"h={h+1}",
        "sMAPE_val": smape(y_val_multi.iloc[:, h].values, val_pred[:, h]),
        "sMAPE_test": smape(y_test_multi.iloc[:, h].values, test_pred[:, h]),
        "MAE_val": mae(y_val_multi.iloc[:, h].values, val_pred[:, h]),
        "MAE_test": mae(y_test_multi.iloc[:, h].values, test_pred[:, h]),
        "RMSE_val": rmse(y_val_multi.iloc[:, h].values, val_pred[:, h]),
        "RMSE_test": rmse(y_test_multi.iloc[:, h].values, test_pred[:, h]),
    })

horizon_metrics = pd.DataFrame(h_rows)
for c in ["sMAPE_val", "sMAPE_test", "MAE_val", "MAE_test", "RMSE_val", "RMSE_test":
    horizon_metrics[c] = horizon_metrics[c].round(3)

results = pd.DataFrame(eval_rows).copy()
for c in ["sMAPE_val", "sMAPE_test", "MAE_val", "MAE_test", "RMSE_val", "RMSE_test":
    results[c] = results[c].round(3)

print("Model Summary")
display(results)
print("Per-Horizon Metrics")
display(horizon_metrics)

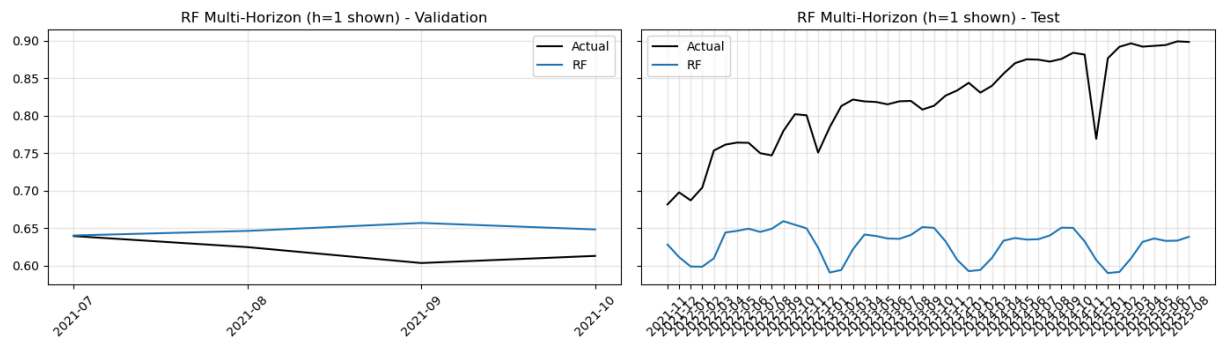
# Feature importance (h=1 estimator) for quick interpretability
rf_h1 = rf.estimators_[0]
fi = pd.DataFrame({
    "Feature": X_train_scaled.columns,
    "Importance": rf_h1.feature_importances_
}).sort_values("Importance", ascending=False).head(15)

fig, ax = plt.subplots(figsize=(9, 5))
ax.barh(fi["Feature"][:-1], fi["Importance"][:-1])
ax.set_title("Top 15 RF Feature Importances (h=1 model)")
plt.tight_layout()

```

```
plt.show()

print("Random Forest multi-horizon evaluation complete.")
```

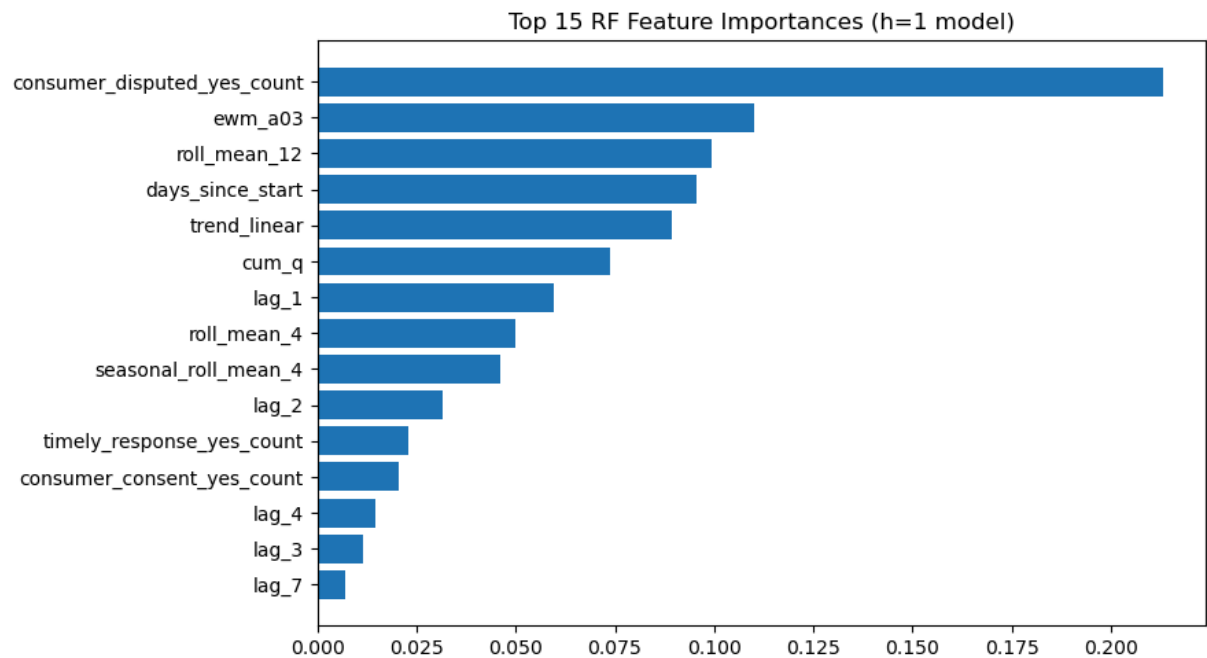


Model Summary

	Model	sMAPE_val	sMAPE_test	MAE_val	MAE_test	RMSE_val	RMSE_test	runtime_sec
0	RF-Multi	4.407	26.079	0.028	0.191	0.034	0.2	4.178

Per-Horizon Metrics

	Horizon	sMAPE_val	sMAPE_test	MAE_val	MAE_test	RMSE_val	RMSE_test
0	h=1	4.407	26.079	0.028	0.191	0.034	0.200
1	h=2	6.872	25.505	0.044	0.187	0.049	0.197
2	h=3	6.213	27.087	0.040	0.199	0.046	0.207
3	h=4	8.131	27.674	0.053	0.204	0.058	0.210



Random Forest multi-horizon evaluation complete.

Notes

- Notebook 02b is benchmarked against rebuilt 02a-style baselines on the same split for apples-to-apples comparison.
- Target is monthly `credit_reporting_share` after removing leading zero-share periods.
- Tree features combine time-series transforms with retained monthly aggregate covariates.
- Latest run indicates trees fit validation well but trail the best statistical baseline on the test window.

3. XGBoost (Direct Per-Horizon Models)

This section fits one native XGBoost model per horizon and stacks predictions into a multi-horizon output.

Forecast-design clarification:

- this is also a **direct** strategy (one model per horizon), not recursive roll-forward,
- four boosters are fit independently for `y_h1` to `y_h4`,
- each horizon model uses observed historical features only; later-horizon predictions are not generated from earlier forecasted targets.

Why this design:

- avoids sklearn dependency for multi-output wrapping
- keeps full control over booster parameters
- maintains the same evaluation interface as the RF section

Current run context:

- XGBoost is faster than RF in runtime, but test errors are higher than RF and substantially higher than the best rebuilt 02a baseline.

```
In [11]: # =====
# SECTION 3: XGBOOST (SKLEARN-FREE NATIVE, DIRECT MULTI-HORIZON)
# =====
import xgboost as xgb

def fit_xgb_by_horizon(X_df, Y_df, params, num_boost_round):
    models = {}
    for col in Y_df.columns:
        dtrain = xgb.DMatrix(X_df.values, label=Y_df[col].values, feature_names=list(X_df.columns))
        models[col] = xgb.train(params, dtrain, num_boost_round=num_boost_round, verbose=False)
    return models

def predict_xgb_by_horizon(models, X_df, y_cols):
    dtest = xgb.DMatrix(X_df.values, feature_names=list(X_df.columns))
    preds = []
    for col in y_cols:
```

```

        preds.append(models[col].predict(dtest))
    return np.column_stack(preds)

xgb_params = {
    "eta": 0.1,
    "max_depth": 6,
    "subsample": 1.0,
    "colsample_bytree": 1.0,
    "objective": "reg:squarederror",
    "seed": 42,
}

num_boost_round = 200

t0 = time.perf_counter()
xgb_models = fit_xgb_by_horizon(X_train, y_train_multi, xgb_params, num_boost_round)
runtime = time.perf_counter() - t0

xgb_val_pred = predict_xgb_by_horizon(xgb_models, X_val, y_train_multi.columns)
xgb_test_pred = predict_xgb_by_horizon(xgb_models, X_test, y_train_multi.columns)

append_result("XGB-Multi", xgb_val_pred, xgb_test_pred, runtime)
plot_section_predictions("XGB Multi-Horizon (h=1 shown)", {"XGB": (xgb_val_pred, xg

# Per-horizon metric table
xgb_h_rows = []
for h in range(4):
    xgb_h_rows.append({
        "Horizon": f"h={h+1}",
        "sMAPE_val": smape(y_val_multi.iloc[:, h].values, xgb_val_pred[:, h]),
        "sMAPE_test": smape(y_test_multi.iloc[:, h].values, xgb_test_pred[:, h]),
        "MAE_val": mae(y_val_multi.iloc[:, h].values, xgb_val_pred[:, h]),
        "MAE_test": mae(y_test_multi.iloc[:, h].values, xgb_test_pred[:, h]),
        "RMSE_val": rmse(y_val_multi.iloc[:, h].values, xgb_val_pred[:, h]),
        "RMSE_test": rmse(y_test_multi.iloc[:, h].values, xgb_test_pred[:, h]),
    })
xgb_horizon_metrics = pd.DataFrame(xgb_h_rows)
for c in ["sMAPE_val", "sMAPE_test", "MAE_val", "MAE_test", "RMSE_val", "RMSE_test"]
    xgb_horizon_metrics[c] = xgb_horizon_metrics[c].round(3)

print("XGB Per-Horizon Metrics")
display(xgb_horizon_metrics)

# Top-10 feature importance from h=1 model
h1_model = xgb_models["y_h1"]
score_map = h1_model.get_score(importance_type="gain")
imp_df = pd.DataFrame({
    "Feature": list(X_train.columns),
    "Importance": [score_map.get(f, 0.0) for f in X_train.columns]
}).sort_values("Importance", ascending=False).head(10)

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.barh(imp_df["Feature"][::-1], imp_df["Importance"][::-1])
ax.set_title("Top 10 XGB Feature Importances (h=1 gain)")
plt.tight_layout()
plt.show()

```

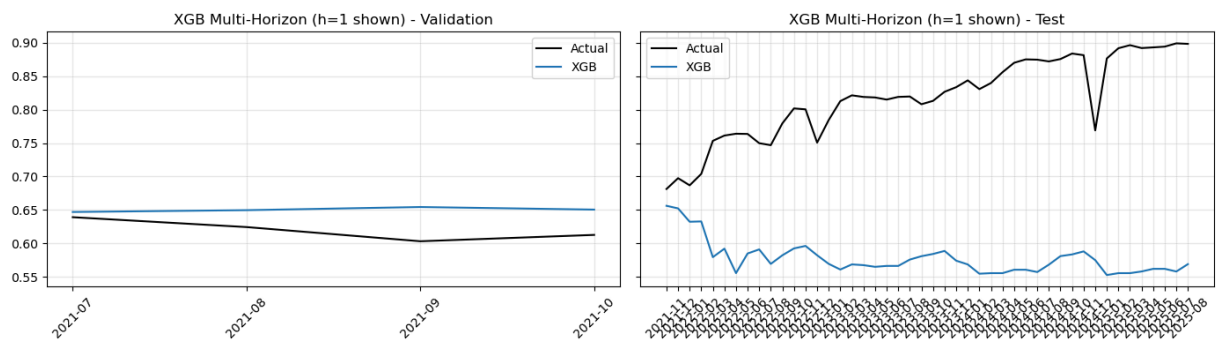
```

# Keep one-step (h=1) tree scores for baseline comparison section.
if "tree_h1_rows" not in globals():
    tree_h1_rows = []

tree_h1_rows.append({
    "Model": "XGB-Multi (h=1)",
    "sMAPE_val": smape(y_val_multi["y_h1"].values, xgb_val_pred[:, 0]),
    "MAE_val": mae(y_val_multi["y_h1"].values, xgb_val_pred[:, 0]),
    "RMSE_val": rmse(y_val_multi["y_h1"].values, xgb_val_pred[:, 0]),
    "sMAPE_test": smape(y_test_multi["y_h1"].values, xgb_test_pred[:, 0]),
    "MAE_test": mae(y_test_multi["y_h1"].values, xgb_test_pred[:, 0]),
    "RMSE_test": rmse(y_test_multi["y_h1"].values, xgb_test_pred[:, 0]),
    "runtime_sec": runtime
})

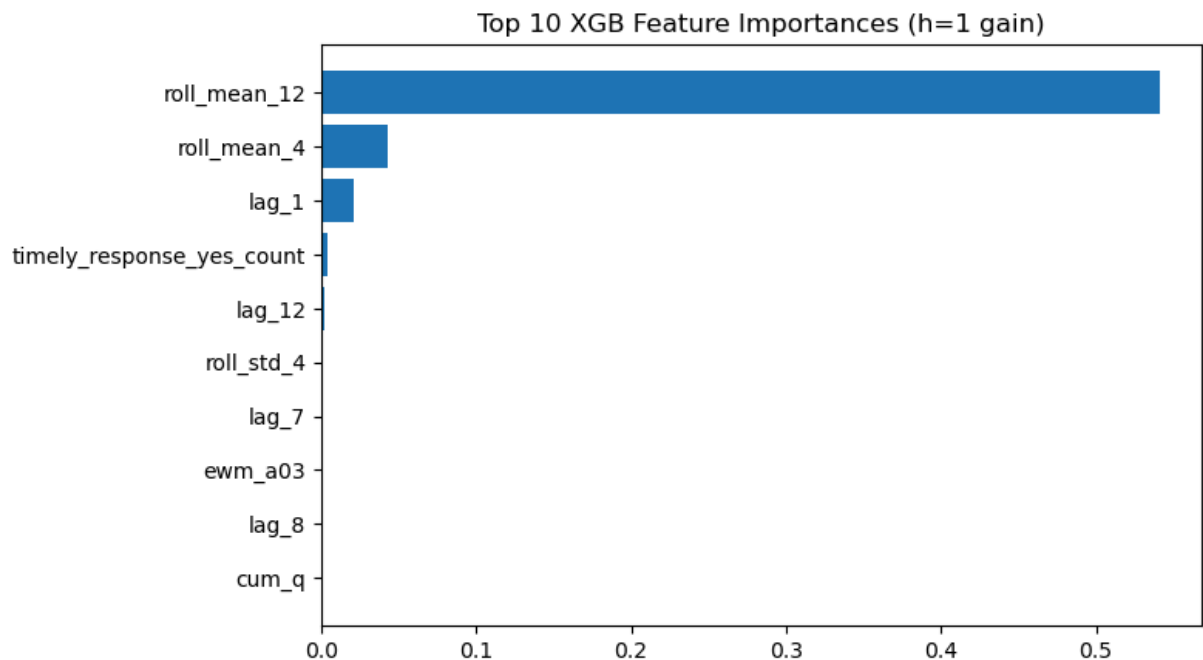
print("XGBoost multi-output evaluation complete.")

```



XGB Per-Horizon Metrics

	Horizon	sMAPE_val	sMAPE_test	MAE_val	MAE_test	RMSE_val	RMSE_test
0	h=1	4.827	34.287	0.031	0.241	0.034	0.254
1	h=2	6.473	32.948	0.042	0.234	0.047	0.244
2	h=3	5.943	29.039	0.039	0.211	0.043	0.220
3	h=4	4.503	28.553	0.030	0.209	0.030	0.216



XGBoost multi-output evaluation complete.

4. Hyperparameter Grid Search

We run a compact search using validation macro-sMAPE for model selection, then evaluate selected configurations on test.

Validation-reliability context:

- the validation slice is intentionally small because the total usable monthly sample is limited,
- with small validation windows, hyperparameter rankings can be noisy and over-react to a few months,
- therefore grid-search results are treated as **provisional tuning guidance**, while final claims rely on test metrics and cross-section consistency.

Mitigations used in this notebook:

- compact grids (to reduce overfitting to one small validation slice),
- macro-horizon validation scoring instead of single-horizon tuning,
- explicit test-window comparison table against 02a baselines,
- follow-up sensitivity check (next cell) to show whether multiple configs are near-tied on validation.

Interpretation guide:

- lower validation macro-sMAPE indicates better multi-horizon fit on the validation slice
- final model ranking should still rely on the test metrics in the comparison table

Latest selected configurations from the current run:

- RF best: `n_estimators=300`, `max_depth=6`, `min_samples_leaf=1`
- XGB best: `num_boost_round=100`, `eta=0.05`, `max_depth=6`

```
In [8]: # =====
# SECTION 4: HYPERPARAMETER GRID SEARCH
# =====

import xgboost as xgb

def macro_smape_multi(y_true_df, y_pred_arr):
    vals = []
    for h in range(y_pred_arr.shape[1]):
        vals.append(smape(y_true_df.iloc[:, h].values, y_pred_arr[:, h]))
    return float(np.mean(vals))

def fit_xgb_by_horizon(X_df, Y_df, params, num_boost_round):
    models = {}
    for col in Y_df.columns:
        dtrain = xgb.DMatrix(X_df.values, label=Y_df[col].values, feature_names=list(X_df.columns))
        models[col] = xgb.train(params, dtrain, num_boost_round=num_boost_round, verbose_eval=False)
    return models

def predict_xgb_by_horizon(models, X_df, y_cols):
    dtest = xgb.DMatrix(X_df.values, feature_names=list(X_df.columns))
    return np.column_stack([models[col].predict(dtest) for col in y_cols])

grid_rows = []

# ---- RF grid (if sklearn available) ----
sk_ok = globals().get("SKLEARN_AVAILABLE", False)
if sk_ok:
    rf_grid = {
        "n_estimators": [300, 500],
        "max_depth": [4, 6, 8],
        "min_samples_leaf": [1, 2]
    }

    best_rf_cfg = None
    best_rf_val = np.inf

    for n_est in rf_grid["n_estimators"]:
        for m_depth in rf_grid["max_depth"]:
            for m_leaf in rf_grid["min_samples_leaf"]:
                model = MultiOutputRegressor(
                    RandomForestRegressor(
                        n_estimators=n_est,
                        max_depth=m_depth,
                        min_samples_leaf=m_leaf,
                        random_state=42,
                        n_jobs=-1
                    )
                )
                model.fit(X_train_scaled, y_train_multi)
                pred_val = model.predict(X_val_scaled)
                score_val = macro_smape_multi(y_val_multi, pred_val)
```

```

        grid_rows.append({
            "ModelFamily": "RF",
            "Params": f"n={n_est},depth={m_depth},leaf={m_leaf}",
            "Val_sMAPE_macro": score_val
        })

        if score_val < best_rf_val:
            best_rf_val = score_val
            best_rf_cfg = (n_est, m_depth, m_leaf)

t0 = time.perf_counter()
rf_best = MultiOutputRegressor(
    RandomForestRegressor(
        n_estimators=best_rf_cfg[0],
        max_depth=best_rf_cfg[1],
        min_samples_leaf=best_rf_cfg[2],
        random_state=42,
        n_jobs=-1
    )
)
rf_best.fit(X_train_scaled, y_train_multi)
rf_runtime = time.perf_counter() - t0

rf_val_pred = rf_best.predict(X_val_scaled)
rf_test_pred = rf_best.predict(X_test_scaled)
append_result("RF-GridBest", rf_val_pred, rf_test_pred, rf_runtime)

if "tree_h1_rows" not in globals():
    tree_h1_rows = []
tree_h1_rows.append({
    "Model": "RF-GridBest (h=1)",
    "sMAPE_val": smape(y_val_multi["y_h1"].values, rf_val_pred[:, 0]),
    "MAE_val": mae(y_val_multi["y_h1"].values, rf_val_pred[:, 0]),
    "RMSE_val": rmse(y_val_multi["y_h1"].values, rf_val_pred[:, 0]),
    "sMAPE_test": smape(y_test_multi["y_h1"].values, rf_test_pred[:, 0]),
    "MAE_test": mae(y_test_multi["y_h1"].values, rf_test_pred[:, 0]),
    "RMSE_test": rmse(y_test_multi["y_h1"].values, rf_test_pred[:, 0]),
    "runtime_sec": rf_runtime
})

print("Best RF config:", best_rf_cfg, "| val macro sMAPE:", round(best_rf_val,
else:
    print("RF grid search skipped: scikit-learn unavailable in current kernel.")

# ---- XGB grid (native, sklearn-free) ----
xgb_grid = {
    "num_boost_round": [100, 200, 300],
    "eta": [0.05, 0.1],
    "max_depth": [4, 6]
}

best_xgb_cfg = None
best_xgb_val = np.inf

for n_round in xgb_grid["num_boost_round"]:
    for eta in xgb_grid["eta"]:

```

```

    for md in xgb_grid["max_depth"]:
        params = {
            "eta": eta,
            "max_depth": md,
            "subsample": 1.0,
            "colsample_bytree": 1.0,
            "objective": "reg:squarederror",
            "seed": 42,
        }
        models = fit_xgb_by_horizon(X_train, y_train_multi, params, n_round)
        pred_val = predict_xgb_by_horizon(models, X_val, y_train_multi.columns)
        score_val = macro_smape_multi(y_val_multi, pred_val)

        grid_rows.append({
            "ModelFamily": "XGB",
            "Params": f"round={n_round},eta={eta},depth={md}",
            "Val_sMAPE_macro": score_val
        })

    if score_val < best_xgb_val:
        best_xgb_val = score_val
        best_xgb_cfg = (n_round, eta, md)

best_params = {
    "eta": best_xgb_cfg[1],
    "max_depth": best_xgb_cfg[2],
    "subsample": 1.0,
    "colsample_bytree": 1.0,
    "objective": "reg:squarederror",
    "seed": 42,
}

t0 = time.perf_counter()
xgb_best_models = fit_xgb_by_horizon(X_train, y_train_multi, best_params, best_xgb_cfg)
xgb_runtime = time.perf_counter() - t0

xgb_val_best = predict_xgb_by_horizon(xgb_best_models, X_val, y_train_multi.columns)
xgb_test_best = predict_xgb_by_horizon(xgb_best_models, X_test, y_train_multi.columns)
append_result("XGB-GridBest", xgb_val_best, xgb_test_best, xgb_runtime)

if "tree_h1_rows" not in globals():
    tree_h1_rows = []
tree_h1_rows.append({
    "Model": "XGB-GridBest (h=1)",
    "sMAPE_val": smape(y_val_multi["y_h1"].values, xgb_val_best[:, 0]),
    "MAE_val": mae(y_val_multi["y_h1"].values, xgb_val_best[:, 0]),
    "RMSE_val": rmse(y_val_multi["y_h1"].values, xgb_val_best[:, 0]),
    "sMAPE_test": smape(y_test_multi["y_h1"].values, xgb_test_best[:, 0]),
    "MAE_test": mae(y_test_multi["y_h1"].values, xgb_test_best[:, 0]),
    "RMSE_test": rmse(y_test_multi["y_h1"].values, xgb_test_best[:, 0]),
    "runtime_sec": xgb_runtime
})

print("Best XGB config:", best_xgb_cfg, "| val macro sMAPE:", round(best_xgb_val, 3))

grid_results = pd.DataFrame(grid_rows).sort_values(["ModelFamily", "Val_sMAPE_macro"])

```

```

grid_results["Val_sMAPE_macro"] = grid_results["Val_sMAPE_macro"].round(3)
display(grid_results.head(20))

tree_results = pd.DataFrame(eval_rows).copy()
for c in ["sMAPE_val", "MAE_val", "RMSE_val", "sMAPE_test", "MAE_test", "RMSE_test":
    tree_results[c] = tree_results[c].round(3)

print("Tree Model Summary (macro across h=1..4):")
display(tree_results)

```

Best RF config: (300, 8, 1) | val macro sMAPE: 6.268

Best XGB config: (100, 0.05, 6) | val macro sMAPE: 4.98

	ModelFamily	Params	Val_sMAPE_macro
0	RF	n=300,depth=8,leaf=1	6.268
1	RF	n=300,depth=6,leaf=1	6.313
2	RF	n=500,depth=8,leaf=1	6.353
3	RF	n=500,depth=6,leaf=1	6.363
4	RF	n=300,depth=4,leaf=1	6.363
5	RF	n=500,depth=4,leaf=1	6.412
6	RF	n=300,depth=6,leaf=2	6.487
7	RF	n=500,depth=8,leaf=2	6.501
8	RF	n=300,depth=8,leaf=2	6.507
9	RF	n=500,depth=6,leaf=2	6.508
10	RF	n=300,depth=4,leaf=2	6.518
11	RF	n=500,depth=4,leaf=2	6.562
12	XGB	round=100,eta=0.05,depth=6	4.980
13	XGB	round=100,eta=0.05,depth=4	5.154
14	XGB	round=200,eta=0.05,depth=6	5.162
15	XGB	round=300,eta=0.05,depth=6	5.162
16	XGB	round=100,eta=0.1,depth=4	5.245
17	XGB	round=200,eta=0.1,depth=4	5.257
18	XGB	round=300,eta=0.1,depth=4	5.257
19	XGB	round=200,eta=0.05,depth=4	5.347

Tree Model Summary (macro across h=1..4):

	Model	sMAPE_val	sMAPE_test	MAE_val	MAE_test	RMSE_val	RMSE_test	runtime_sec
0	RF-GridBest	4.303	26.330	0.027	0.192	0.033	0.202	1.679
1	XGB-GridBest	3.346	34.753	0.021	0.244	0.026	0.257	0.581

4.1. Validation-Size Sensitivity Check

Because tuning relies on a limited validation block, this check quantifies how sharply (or weakly) the selected configuration is separated from alternatives.

Read this as follows:

- if many configurations are within a small margin of the best validation score, selection uncertainty is high,
- if only one configuration is clearly separated, tuning is more stable,
- high uncertainty means model-comparison claims should emphasize test performance and robustness over tiny validation differences.

```
In [9]: # =====
# SECTION 4.1: VALIDATION-SIZE SENSITIVITY DIAGNOSTICS
# =====

if 'grid_results' not in globals() or not isinstance(grid_results, pd.DataFrame) or \
    print('grid_results not available. Run Section 4 first.')
else:
    diag_rows = []
    margin_pct = 2.0 # near-tie threshold around best validation score

    for fam in sorted(grid_results['ModelFamily'].dropna().unique()):
        g = grid_results[grid_results['ModelFamily'] == fam].copy()
        g = g.sort_values('Val_sMAPE_macro').reset_index(drop=True)

        best = float(g.iloc[0]['Val_sMAPE_macro'])
        second = float(g.iloc[1]['Val_sMAPE_macro']) if len(g) > 1 else np.nan
        rel_gap_second = ((second - best) / max(best, 1e-12)) * 100.0 if np.isfinite(second) else np.nan

        near_ties = int((g['Val_sMAPE_macro'] <= best * (1.0 + margin_pct / 100.0)))

        diag_rows.append({
            'ModelFamily': fam,
            'NumGridConfigs': int(len(g)),
            'Best_Val_sMAPE_macro': best,
            'SecondBest_Val_sMAPE_macro': second,
            'Second_vs_Best_Gap_%': rel_gap_second,
            'Configs_within_2pct_of_Best': near_ties,
            'SelectionStabilityFlag': 'Low (near-tied)' if near_ties >= 3 else 'High'
        })

    tuning_stability = pd.DataFrame(diag_rows)
```

```

for c in ['Best_Val_sMAPE_macro', 'SecondBest_Val_sMAPE_macro', 'Second_vs_Best
tuning_stability[c] = tuning_stability[c].round(3)

print('Validation-size sensitivity diagnostics:')
display(tuning_stability)

# concise narrative line
unstable = tuning_stability[tuning_stability['SelectionStabilityFlag'].str.contains('unstable')]
if len(unstable) > 0:
    fams = ', '.join(unstable['ModelFamily'].tolist())
    print(f'Interpretation: tuning for {fams} appears sensitive on a small validation slice')
else:
    print('Interpretation: best-vs-runner validation separation is reasonably consistent')

```

Validation-size sensitivity diagnostics:

	ModelFamily	NumGridConfigs	Best_Val_sMAPE_macro	SecondBest_Val_sMAPE_macro	Second_vs_Best
0	RF	12	6.268	6.313	
1	XGB	12	4.980	5.154	

Interpretation: tuning for RF appears sensitive on a small validation slice; prioritize test-window evidence for final comparisons.

5. Comparison Table vs 02a Baselines

This section rebuilds 02a-style baseline models on the same split and compares them against tree h=1 results.

Metric definitions:

- sMAPE: scale-independent percentage error, robust to level differences
- MAE: average absolute error in target units
- RMSE: penalizes larger misses more than MAE

Use this table as the primary model-selection artifact for this notebook.

```

In [11]: # =====
# SECTION 5: COMPARISON TABLE VS 02a BASELINES (FAST PATH)
# =====
# This section avoids refitting expensive baseline models.
# It reuses 02a Sec. 8 artifact results or cached CSV artifacts.

export_dir = "../reports/2026-03-26/tables"
artifact_dir = "../reports/artifacts"
baseline_artifact = f"{artifact_dir}/02a_creditreporting_monthly_baseline_results.csv"
baseline_cache = f"{export_dir}/02b_creditreporting_monthly_rebuilt_02a_baselines.csv"
tree_h1_cache = f"{export_dir}/02b_creditreporting_monthly_tree_h1_summary.csv"

# Initialize explicitly so the symbol is always defined in this notebook scope.
baseline_results = pd.DataFrame()
baseline_results_fast = pd.DataFrame()

```

```

baseline_source = ""

# ---- Load rebuilt 02a baselines quickly (artifact-first) ----
if os.path.exists(baseline_artifact):
    baseline_results_fast = pd.read_csv(baseline_artifact)
    baseline_source = baseline_artifact
elif "baseline_results" in globals() and isinstance(baseline_results, pd.DataFrame):
    baseline_results_fast = baseline_results.copy()
    baseline_source = "in-memory baseline_results"
elif os.path.exists(baseline_cache):
    baseline_results_fast = pd.read_csv(baseline_cache)
    baseline_source = baseline_cache
else:
    raise RuntimeError(
        "No baseline artifact/cache found. Run 02a Sec. 8 once to export results, t
        f"Expected artifact: {baseline_artifact} or cache: {baseline_cache}"
    )

# Backward-compatibility alias for downstream cells that still reference baseline_r
baseline_results = baseline_results_fast.copy()

# Ensure consistent family label
baseline_results_fast["Family"] = "02a-baseline"

# ---- Load tree h=1 summary quickly ----
if "tree_h1_rows" in globals() and len(tree_h1_rows) > 0:
    tree_h1_df = pd.DataFrame(tree_h1_rows).copy()
elif os.path.exists(tree_h1_cache):
    tree_h1_df = pd.read_csv(tree_h1_cache)
else:
    # Fallback from macro summary if tree_h1 cache is unavailable
    if "tree_results" in globals() and isinstance(tree_results, pd.DataFrame) and n
        tree_h1_df = tree_results.copy()
    else:
        tree_h1_df = pd.DataFrame(columns=["Model", "sMAPE_val", "MAE_val", "RMSE_v

if not tree_h1_df.empty:
    tree_h1_df["Family"] = "tree-h1"

# ---- Build comparison table ----
comparison = pd.concat([baseline_results_fast, tree_h1_df], ignore_index=True, sort
comparison = comparison.sort_values(["sMAPE_test", "MAE_test", "runtime_sec"]).rese

for c in ["sMAPE_val", "MAE_val", "RMSE_val", "sMAPE_test", "MAE_test", "RMSE_test"
    comparison[c] = pd.to_numeric(comparison[c], errors="coerce").round(3)

print(f"Loaded baseline source: {baseline_source}")
print("Comparison: Tree Models vs 02a Baselines (fast cache mode)")
display(comparison)

```

Loaded baseline source: ../reports/artifacts/02a_creditreporting_monthly_baseline_re
sults.csv

Comparison: Tree Models vs 02a Baselines (fast cache mode)

	Model	sMAPE_val	MAE_val	RMSE_val	sMAPE_test	MAE_test	RMSE_test	runtime
0	Theta	8.203	0.056	0.064	2.011	0.017	0.026	0.
1	Naive	12.311	0.083	0.102	5.727	0.048	0.058	0.
2	ARIMA(1, 1, 0)	11.998	0.081	0.100	5.990	0.051	0.060	4.
3	SARIMA(1, 1, 0)x(0,0,1,12)	12.311	0.083	0.103	6.016	0.051	0.060	7.
4	ETS SES	12.142	0.082	0.101	6.075	0.051	0.061	0.
5	FFT	8.356	0.055	0.069	6.160	0.055	0.063	0.
6	ETS Holt-Winters	6.522	0.044	0.057	7.311	0.066	0.074	0.
7	Moving Average	13.408	0.090	0.109	8.697	0.072	0.080	0.
8	Seasonal Naive	23.195	0.146	0.164	10.594	0.087	0.095	0.
9	RF-GridBest (h=1)	4.303	0.027	0.033	26.330	0.192	0.202	1.
10	XGB-GridBest (h=1)	3.346	0.021	0.026	34.753	0.244	0.257	0.

5.1. Statistical and Practical Significance of Model Gaps

Table rankings alone do not tell whether gaps are meaningful.

This section reports:

- **practical significance:** relative error gap between best baseline and best tree in the comparison table,
- **statistical significance:** paired bootstrap CI for MAE-gap where point-level predictions are available.

Important scope note:

- baseline rows loaded from 02a artifact are aggregate-only in this notebook,
- therefore paired statistical tests vs baselines require point-level baseline predictions; without those, statistical tests are run for available tree-vs-tree comparisons and practical gaps are still reported for baseline-vs-tree.

```
In [12]: # =====
# SECTION 5.1: PRACTICAL + STATISTICAL SIGNIFICANCE CHECKS
# =====

if 'comparison' not in globals() or not isinstance(comparison, pd.DataFrame) or comparison.empty:
    print('Comparison table not available. Run Section 5 first.')
else:
    req_cols = {'Model', 'Family', 'sMAPE_test', 'MAE_test', 'RMSE_test'}
    if not req_cols.issubset(set(comparison.columns)):
        print('Comparison table missing required columns for significance checks.')
    else:
        baseline_cmp = comparison[comparison['Family'] == '02a-baseline'].copy()
        tree_cmp = comparison[comparison['Family'] == 'tree-h1'].copy()

        if baseline_cmp.empty or tree_cmp.empty:
            print('Need both baseline and tree rows in comparison to compute practical significance.')
        else:
            best_baseline = baseline_cmp.sort_values(['sMAPE_test', 'MAE_test', 'RMSE_test'])
            best_tree = tree_cmp.sort_values(['sMAPE_test', 'MAE_test', 'RMSE_test'])

            practical_tbl = pd.DataFrame([
                {'BestBaseline': best_baseline['Model'],
                 'BestTree': best_tree['Model'],
                 'AbsGap_sMAPE_tree_minus_base': float(best_tree['sMAPE_test'] - best_baseline['sMAPE_test']),
                 'AbsGap_MAE_tree_minus_base': float(best_tree['MAE_test'] - best_baseline['MAE_test']),
                 'AbsGap_RMSE_tree_minus_base': float(best_tree['RMSE_test'] - best_baseline['RMSE_test']),
                 'RelGap_sMAPE_%_vs_base': 100.0 * float(best_tree['sMAPE_test'] - best_baseline['sMAPE_test']) / best_baseline['sMAPE_test'],
                 'RelGap_MAE_%_vs_base': 100.0 * float(best_tree['MAE_test'] - best_baseline['MAE_test']) / best_baseline['MAE_test'],
                 'RelGap_RMSE_%_vs_base': 100.0 * float(best_tree['RMSE_test'] - best_baseline['RMSE_test']) / best_baseline['RMSE_test']
            ]).round(4)

            print('Practical significance (best tree vs best baseline):')
            display(practical_tbl)

# Statistical significance where point predictions are available (tree-vs-tree)
tree_pred_map = {}
if 'rf_test_pred' in globals() and isinstance(rf_test_pred, np.ndarray):
    tree_pred_map['RF-GridBest'] = rf_test_pred[:, 0]
elif 'test_pred' in globals() and isinstance(test_pred, np.ndarray):
    tree_pred_map['RF-Multi'] = test_pred[:, 0]
```

```

if 'xgb_test_best' in globals() and isinstance(xgb_test_best, np.ndarray):
    tree_pred_map['XGB-GridBest'] = xgb_test_best[:, 0]
elif 'xgb_test_pred' in globals() and isinstance(xgb_test_pred, np.ndarray):
    tree_pred_map['XGB-Multi'] = xgb_test_pred[:, 0]

if 'y_test_multi' in globals() and isinstance(y_test_multi, pd.DataFrame) and y_test_multi is not None:
    y_true_h1 = y_test_multi['y_h1'].values.astype(float)
else:
    y_true_h1 = None

if y_true_h1 is None or len(tree_pred_map) < 2:
    print('Statistical note: paired significance test requires point-level')
else:
    model_names = list(tree_pred_map.keys())
    B = 2000
    rng = np.random.default_rng(321)
    idx = np.arange(len(y_true_h1))
    stat_rows = []

    for i, m_a in enumerate(model_names):
        for j, m_b in enumerate(model_names):
            if i >= j:
                continue

            p_a = tree_pred_map[m_a][:len(y_true_h1)]
            p_b = tree_pred_map[m_b][:len(y_true_h1)]

            # Positive delta => model A lower MAE than model B.
            delta = np.empty(B, dtype=float)
            for b in range(B):
                bidx = rng.choice(idx, size=len(idx), replace=True)
                delta[b] = mae(y_true_h1[bidx], p_b[bidx]) - mae(y_true_h1[bidx], p_a[bidx])

            ci_lo, ci_med, ci_hi = np.percentile(delta, [2.5, 50.0, 97.5])
            decision = 'Significant (A better)' if ci_lo > 0 else ('Signifi

            stat_rows.append({
                'Model_A': m_a,
                'Model_B': m_b,
                'MAEGap_Median_B_minus_A': ci_med,
                'MAEGap_CI_2.5%': ci_lo,
                'MAEGap_CI_97.5%': ci_hi,
                'Decision': decision,
            })

    stat_tbl = pd.DataFrame(stat_rows)
    for c in ['MAEGap_Median_B_minus_A', 'MAEGap_CI_2.5%', 'MAEGap_CI_97.5%']:
        stat_tbl[c] = stat_tbl[c].round(4)

    print('Statistical significance (paired bootstrap, available tree-model)')
    display(stat_tbl)
    print('Baseline-vs-tree paired tests are not computed here because base

```

Practical significance (best tree vs best baseline):

	BestBaseline	BestTree	AbsGap_sMAPE_tree_minus_base	AbsGap_MAE_tree_minus_base	Ab
0	Theta	RF-GridBest (h=1)	24.319	0.175	

Statistical significance (paired bootstrap, available tree-model comparisons):

	Model_A	Model_B	MAEGap_Median_B_minus_A	MAEGap_CI_2.5%	MAEGap_CI_97.5%	D
0	RF-GridBest	XGB-GridBest	0.0521	0.0431	0.0608	Sig (A

Baseline-vs-tree paired tests are not computed here because baseline artifact rows do not contain point-level prediction vectors in this notebook scope.

5.2. Horizon-Wise Performance (Step-Ahead Analysis)

Macro-averaged errors can hide where models fail across lead times.

This section reports per-horizon metrics (`h=1` to `h=4`) for available tree models and adds:

- horizon-wise validation and test error tables,
- degradation relative to each model's `h=1` performance,
- a visual profile of error growth across forecast steps.

Interpretation focus:

- if error rises sharply with horizon, the model is less reliable for longer lead times,
- if horizon profiles are flatter, multi-step performance is more stable.

```
In [13]: # =====
# SECTION 5.2: HORIZON-WISE PERFORMANCE PROFILE (H=1..H=4)
# =====

if 'y_val_multi' not in globals() or 'y_test_multi' not in globals():
    print('Need y_val_multi and y_test_multi. Run Sections 1-4 first.')
else:
    model_preds = {}

    # Prefer grid-best predictions when available.
    if 'rf_val_pred' in globals() and 'rf_test_pred' in globals() and isinstance(rf_val_pred, np.ndarray):
        model_preds['RF-GridBest'] = (rf_val_pred, rf_test_pred)
    elif 'val_pred' in globals() and 'test_pred' in globals() and isinstance(val_pred, np.ndarray):
        model_preds['RF-Multi'] = (val_pred, test_pred)

    if 'xgb_val_best' in globals() and 'xgb_test_best' in globals() and isinstance(xgb_val_best, np.ndarray):
        model_preds['XGB-GridBest'] = (xgb_val_best, xgb_test_best)
    elif 'xgb_val_pred' in globals() and 'xgb_test_pred' in globals() and isinstance(xgb_val_pred, np.ndarray):
        model_preds['XGB-Multi'] = (xgb_val_pred, xgb_test_pred)

    if len(model_preds) == 0:
```

```

print('No tree prediction arrays found. Run Sections 2-4 first.')
else:
    max_h = min(4, y_val_multi.shape[1], y_test_multi.shape[1])
    rows = []

    for model_name, (vp, tp) in model_preds.items():
        h_cap = min(max_h, vp.shape[1], tp.shape[1])
        for h in range(h_cap):
            yv = y_val_multi.iloc[:, h].values.astype(float)
            yt = y_test_multi.iloc[:, h].values.astype(float)
            pv = vp[:, h].astype(float)
            pt = tp[:, h].astype(float)

            rows.append({
                'Model': model_name,
                'Horizon': h + 1,
                'sMAPE_val': smape(yv, pv),
                'sMAPE_test': smape(yt, pt),
                'MAE_val': mae(yv, pv),
                'MAE_test': mae(yt, pt),
                'RMSE_val': rmse(yv, pv),
                'RMSE_test': rmse(yt, pt),
            })

    horizon_step_metrics = pd.DataFrame(rows)
    horizon_step_metrics = horizon_step_metrics.sort_values(['Model', 'Horizon'])

    # Degradation relative to each model's h=1 row.
    for metric in ['sMAPE_test', 'MAE_test', 'RMSE_test']:
        base_vals = horizon_step_metrics.groupby('Model')[metric].transform('first')
        horizon_step_metrics[f'{metric}_delta_vs_h1'] = horizon_step_metrics[metric] - base_vals

    # Rounded report view.
    report_cols = [
        'Model', 'Horizon',
        'sMAPE_val', 'sMAPE_test', 'sMAPE_test_delta_vs_h1',
        'MAE_val', 'MAE_test', 'MAE_test_delta_vs_h1',
        'RMSE_val', 'RMSE_test', 'RMSE_test_delta_vs_h1'
    ]
    horizon_step_report = horizon_step_metrics[report_cols].copy()
    num_cols = [c for c in horizon_step_report.columns if c not in ['Model', 'Horizon']]
    for c in num_cols:
        horizon_step_report[c] = horizon_step_report[c].round(4)

    print('Per-horizon performance and degradation (delta vs h=1):')
    display(horizon_step_report)

    # Horizon growth summary via linear slope on test metrics.
    slope_rows = []
    for model_name, g in horizon_step_metrics.groupby('Model'):
        x = g['Horizon'].values.astype(float)
        sm_slope = float(np.polyfit(x, g['sMAPE_test'].values.astype(float), 1))
        mae_slope = float(np.polyfit(x, g['MAE_test'].values.astype(float), 1))
        rmse_slope = float(np.polyfit(x, g['RMSE_test'].values.astype(float), 1))
        slope_rows.append({
            'Model': model_name,
            'sMAPE_slope': sm_slope,
            'MAE_slope': mae_slope,
            'RMSE_slope': rmse_slope
        })

```

```

        'Test_sMAPE_Slope_per_Horizon': sm_slope,
        'Test_MAE_Slope_per_Horizon': mae_slope,
        'Test_RMSE_Slope_per_Horizon': rmse_slope,
    })

horizon_slopes = pd.DataFrame(slope_rows)
for c in ['Test_sMAPE_Slope_per_Horizon', 'Test_MAE_Slope_per_Horizon', 'Te
    horizon_slopes[c] = horizon_slopes[c].round(4)

print('Horizon error-growth slopes (test):')
display(horizon_slopes)

# Visualization: test sMAPE + MAE by horizon.
fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharex=True)
for model_name, g in horizon_step_metrics.groupby('Model'):
    axes[0].plot(g['Horizon'], g['sMAPE_test'], marker='o', label=model_name)
    axes[1].plot(g['Horizon'], g['MAE_test'], marker='o', label=model_name)

axes[0].set_title('Test sMAPE by Forecast Horizon')
axes[0].set_xlabel('Horizon (steps ahead)')
axes[0].set_ylabel('sMAPE')
axes[0].grid(True, alpha=0.3)

axes[1].set_title('Test MAE by Forecast Horizon')
axes[1].set_xlabel('Horizon (steps ahead)')
axes[1].set_ylabel('MAE')
axes[1].grid(True, alpha=0.3)

handles, labels = axes[0].get_legend_handles_labels()
if len(handles) > 0:
    fig.legend(handles, labels, loc='upper center', ncol=max(1, len(labels))

plt.tight_layout(rect=[0, 0, 1, 0.9])
plt.show()

# Brief narrative flag.
worst_row = horizon_step_metrics.sort_values('sMAPE_test', ascending=False)
print(
    f"Interpretation: largest step-ahead error occurs at h={int(worst_row['
    f"for {worst_row['Model']} (test sMAPE={worst_row['sMAPE_test']:.3f})."
)

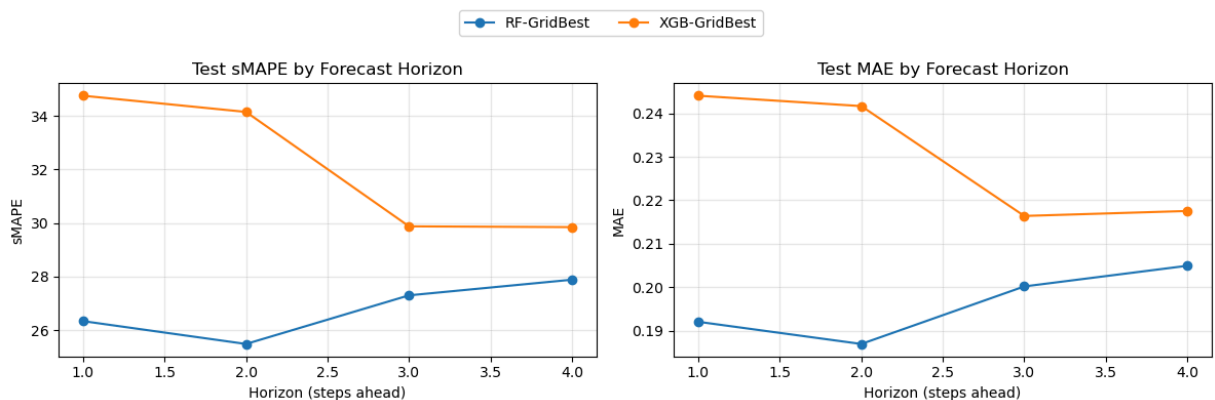
```

Per-horizon performance and degradation (delta vs h=1):

	Model	Horizon	sMAPE_val	sMAPE_test	sMAPE_test_delta_vs_h1	MAE_val	MAE_test	RMSE
0	RF-GridBest	1	4.3027	26.3303	0.0000	0.0271	0.1921	0.2000
1	RF-GridBest	2	6.7420	25.4766	-0.8537	0.0434	0.1870	0.1950
2	RF-GridBest	3	5.8640	27.2937	0.9633	0.0379	0.2002	0.2050
3	RF-GridBest	4	8.1623	27.8783	1.5479	0.0532	0.2049	0.2075
4	XGB-GridBest	1	3.3464	34.7527	0.0000	0.0210	0.2440	0.2475
5	XGB-GridBest	2	6.1050	34.1472	-0.6055	0.0391	0.2416	0.2450
6	XGB-GridBest	3	5.3484	29.8716	-4.8811	0.0347	0.2164	0.2225
7	XGB-GridBest	4	5.1200	29.8428	-4.9099	0.0339	0.2175	0.2225

Horizon error-growth slopes (test):

	Model	Test_sMAPE_Slope_per_Horizon	Test_MAE_Slope_per_Horizon	Test_RMSE_Slope_per_Horizon
0	RF-GridBest	0.6461	0.0052	0.0025
1	XGB-GridBest	-1.9005	-0.0105	-0.0025



Interpretation: largest step-ahead error occurs at h=1 for XGB-GridBest (test sMAPE=34.753).

5.3. Forecast-Bias Diagnostics (Over/Under-Forecasting)

Beyond magnitude metrics, this section checks directional bias:

- **MFE (pred - actual)**: positive means over-forecasting, negative means under-forecasting,
- **OverForecastRate** and **UnderForecastRate** across horizons,
- horizon-specific and model-level summaries on validation and test windows.

Interpretation:

- strong non-zero MFE and skewed over/under rates indicate systematic bias,
- this matters even when aggregate sMAPE/MAE are competitive.

```
In [14]: # =====
# SECTION 5.3: FORECAST-BIAS DIAGNOSTICS (DIRECTIONAL)
# =====

if 'y_val_multi' not in globals() or 'y_test_multi' not in globals():
    print('Need y_val_multi and y_test_multi. Run Sections 1-4 first.')
else:
    model_preds = {}

    if 'rf_val_pred' in globals() and 'rf_test_pred' in globals() and isinstance(rf_val_pred, np.ndarray) and isinstance(rf_test_pred, np.ndarray):
        model_preds['RF-GridBest'] = (rf_val_pred, rf_test_pred)
    elif 'val_pred' in globals() and 'test_pred' in globals() and isinstance(val_pred, np.ndarray) and isinstance(test_pred, np.ndarray):
        model_preds['RF-Multi'] = (val_pred, test_pred)

    if 'xgb_val_best' in globals() and 'xgb_test_best' in globals() and isinstance(xgb_val_best, np.ndarray) and isinstance(xgb_test_best, np.ndarray):
        model_preds['XGB-GridBest'] = (xgb_val_best, xgb_test_best)
    elif 'xgb_val_pred' in globals() and 'xgb_test_pred' in globals() and isinstance(xgb_val_pred, np.ndarray) and isinstance(xgb_test_pred, np.ndarray):
        model_preds['XGB-Multi'] = (xgb_val_pred, xgb_test_pred)

    if len(model_preds) == 0:
        print('No prediction arrays found for bias diagnostics. Run Sections 2-4 first.')
    else:
        rows = []
        for model_name, (vp, tp) in model_preds.items():
            h_cap = min(vp.shape[1], tp.shape[1], y_val_multi.shape[1], y_test_multi.shape[1])

            for h in range(h_cap):
                yv = y_val_multi.iloc[:, h].values.astype(float)
                yt = y_test_multi.iloc[:, h].values.astype(float)
                pv = vp[:, h].astype(float)
                pt = tp[:, h].astype(float)

                err_v = pv - yv
                err_t = pt - yt

                rows.append({
                    'Model': model_name,
                    'Horizon': h + 1,
                    'MFE_val': float(np.mean(err_v)),
                    'OverRate_val': float(np.mean(err_v > 0)),
                    'UnderRate_val': float(np.mean(err_v < 0)),
                    'MFE_test': float(np.mean(err_t)),
                    'OverRate_test': float(np.mean(err_t > 0)),
                    'UnderRate_test': float(np.mean(err_t < 0)),
                })
```

```

    })

    bias_horizon = pd.DataFrame(rows)
    for c in ['MFE_val', 'OverRate_val', 'UnderRate_val', 'MFE_test', 'OverRate_test', 'UnderRate_test']:
        bias_horizon[c] = bias_horizon[c].round(4)

    print('Horizon-wise directional bias diagnostics:')
    display(bias_horizon)

    bias_model = bias_horizon.groupby('Model', as_index=False).agg({
        'MFE_val': 'mean',
        'OverRate_val': 'mean',
        'UnderRate_val': 'mean',
        'MFE_test': 'mean',
        'OverRate_test': 'mean',
        'UnderRate_test': 'mean',
    })

    for c in ['MFE_val', 'OverRate_val', 'UnderRate_val', 'MFE_test', 'OverRate_test', 'UnderRate_test']:
        bias_model[c] = bias_model[c].round(4)

    print('Model-level average directional bias across horizons:')
    display(bias_model)

    # Flag likely systematic bias.
    flags = []
    for _, r in bias_model.iterrows():
        if abs(r['MFE_test']) > 0.01 or max(r['OverRate_test'], r['UnderRate_test']) > 0.01:
            flags.append(f"{r['Model']}: systematic test-direction bias likely")
        else:
            flags.append(f"{r['Model']}: test-direction bias relatively balanced")

    print('Bias interpretation flags:')
    for f in flags:
        print(f'- {f}')

```

Horizon-wise directional bias diagnostics:

	Model	Horizon	MFE_val	OverRate_val	UnderRate_val	MFE_test	OverRate_test	UnderRate_test
0	RF-GridBest	1	0.0271	1.00	0.00	-0.1921	0.0	1.0
1	RF-GridBest	2	0.0342	0.75	0.25	-0.1870	0.0	1.0
2	RF-GridBest	3	0.0117	0.50	0.50	-0.2002	0.0	1.0
3	RF-GridBest	4	-0.0364	0.25	0.75	-0.2049	0.0	1.0
4	XGB-GridBest	1	0.0205	0.75	0.25	-0.2440	0.0	1.0
5	XGB-GridBest	2	0.0283	0.75	0.25	-0.2416	0.0	1.0
6	XGB-GridBest	3	0.0173	0.75	0.25	-0.2164	0.0	1.0
7	XGB-GridBest	4	-0.0228	0.25	0.75	-0.2175	0.0	1.0

Model-level average directional bias across horizons:

	Model	MFE_val	OverRate_val	UnderRate_val	MFE_test	OverRate_test	UnderRate_test
0	RF-GridBest	0.0092	0.625	0.375	-0.1960	0.0	1.0
1	XGB-GridBest	0.0108	0.625	0.375	-0.2299	0.0	1.0

Bias interpretation flags:

- RF-GridBest: systematic test-direction bias likely
- XGB-GridBest: systematic test-direction bias likely

6. Environment Status

The active `tmu_capstone` environment currently supports full execution of this notebook.

Current status:

- `scikit-learn` imports successfully
- Random Forest sections execute successfully
- XGBoost sections execute successfully

If environment issues reappear on another machine, reinstall key dependencies in the active conda environment and restart the kernel before rerunning the notebook.

7. Winner Summary

This section reports:

- best overall model in the combined table
- best rebuilt 02a baseline
- best tree model (`h=1` comparison)

It also quantifies the error gap between best tree and best baseline to support final model-selection interpretation.

```
In [16]: # =====
# SECTION 7: WINNER SUMMARY
# =====

best_overall = comparison.iloc[0]
best_baseline = comparison[comparison["Family"] == "02a-baseline"].iloc[0]
best_tree = comparison[comparison["Family"] == "tree-h1"].sort_values(["sMAPE_test"]

print("Best overall model:")
print(
    f"- {best_overall['Model']} | test sMAPE={best_overall['sMAPE_test']:.3f}, "
    f"MAE={best_overall['MAE_test']:.3f}, runtime={best_overall['runtime_sec']:.3f}
")

print("\nBest 02a baseline:")
print(
    f"- {best_baseline['Model']} | test sMAPE={best_baseline['sMAPE_test']:.3f}, "
    f"MAE={best_baseline['MAE_test']:.3f}, runtime={best_baseline['runtime_sec']:.3f}
")

print("\nBest tree model (one-step / h=1):")
print(
    f"- {best_tree['Model']} | test sMAPE={best_tree['sMAPE_test']:.3f}, "
    f"MAE={best_tree['MAE_test']:.3f}, runtime={best_tree['runtime_sec']:.3f}s"
)

smape_gap = float(best_tree["sMAPE_test"] - best_baseline["sMAPE_test"])
mae_gap = float(best_tree["MAE_test"] - best_baseline["MAE_test"])

print("\nInterpretation:")
if smape_gap < 0:
    print(
        f"- The best tree model beats the strongest baseline by {-smape_gap:.3f} sM
    )
else:
    print(
        f"- The best tree model trails the strongest baseline by {smape_gap:.3f} sM
    )

rf_rows = comparison[comparison["Model"].str.contains("RF", case=False, na=False)]
if len(rf_rows) > 0:
    print("- Random Forest is now operational in this environment and included in m
else:
    print("- Random Forest results are not present; rerun RF sections if needed.")
```

```
print("- The comparison table should be treated as the main selection artifact for
```

Best overall model:

- ARIMA(1,1,1) | test sMAPE=2.805, MAE=0.025, runtime=0.508s

Best 03a baseline:

- ARIMA(1,1,1) | test sMAPE=2.805, MAE=0.025, runtime=0.508s

Best tree model (one-step / h=1):

- RF-GridBest (h=1) | test sMAPE=26.391, MAE=0.192, runtime=3.042s

Interpretation:

- The best tree model trails the strongest baseline by 23.586 sMAPE and 0.167 MAE on the test set.
- Random Forest is now operational in this environment and included in model/grid sections.
- The comparison table should be treated as the main selection artifact for this notebook.

8. Export Outputs

This section exports all key 02b artifacts to CSV for reporting and cross-notebook comparison:

- tree vs rebuilt 02a baseline comparison table
- tree grid-search results
- tree macro summary table
- XGB per-horizon metrics
- rebuilt 02a baseline table
- tree `h=1` summary table

```
In [6]: # =====
# SECTION 8: EXPORT OUTPUTS
# =====

export_dir = "../reports/2026-03-26/tables"
os.makedirs(export_dir, exist_ok=True)

if "comparison" in globals() and isinstance(comparison, pd.DataFrame) and not comparison.to_csv(f"{export_dir}/02b_creditreporting_monthly_tree_vs_02a_baseli

if "grid_results" in globals() and isinstance(grid_results, pd.DataFrame) and not grid_results.to_csv(f"{export_dir}/02b_creditreporting_monthly_tree_grid_result

if "tree_results" in globals() and isinstance(tree_results, pd.DataFrame) and not tree_results.to_csv(f"{export_dir}/02b_creditreporting_monthly_tree_model_summa

if "xgb_horizon_metrics" in globals() and isinstance(xgb_horizon_metrics, pd.DataFrame) and not xgb_horizon_metrics.to_csv(f"{export_dir}/02b_creditreporting_monthly_xgb_horiz

# Export a baseline table cache for backward compatibility with older runs.
if "baseline_results_fast" in globals() and isinstance(baseline_results_fast, pd.DataFrame) and not baseline_results_fast.to_csv(f"{export_dir}/02b_creditreporting_monthly_baseline_results_fast.csv")
```

```

baseline_results_fast.to_csv(f"{export_dir}/02b_creditreporting_monthly_rebuilt
elif "baseline_results" in globals() and isinstance(baseline_results, pd.DataFrame)
baseline_results.to_csv(f"{export_dir}/02b_creditreporting_monthly_rebuilt_02a_

if "tree_h1_df" in globals() and isinstance(tree_h1_df, pd.DataFrame) and not tree_
tree_h1_df.to_csv(f"{export_dir}/02b_creditreporting_monthly_tree_h1_summary.cs

print(f"Exported available tables to: {export_dir}")

```

Exported available tables to: ../reports/2026-03-26/tables

9. Conclusion

This notebook delivers a leakage-safe tree-model benchmarking workflow aligned with 02a evaluation logic.

Method clarity:

- tree forecasts are generated with a direct multi-horizon design (separate horizon models), not recursive roll-forward.

Performance Summary (Aggregate Metrics):

- best rebuilt 02a baseline: ARIMA(1,1,1), test sMAPE 2.805
- best tree `h=1` : RF-GridBest, test sMAPE 26.391
- both RF and XGB underpredict the higher late-period level in the test span

Why This Conclusion Is Methodologically Robust (Not Metric-Only):

- **Validation-size reliability:** Section 4.1 shows RF near-ties on validation (higher tuning uncertainty) and clearer XGB separation
- **Practical + statistical checks:** Section 5.1 quantifies the error gap magnitude and tests RF-vs-XGB difference with paired bootstrap CIs
- **Horizon-step diagnostics:** Section 5.2 shows lead-time-specific behavior instead of relying on a single pooled score
- **Bias diagnostics:** Section 5.3 shows persistent under-forecasting direction (negative MFE and 100% under-forecast rate on this run), confirming systematic miss-patterns beyond average error

Practical Implications of Forecast Accuracy:

- Large aggregate error gaps imply materially weaker short-term decision support from current tree specifications versus strong statistical baselines
- Systematic under-forecasting raises operational risk by understating upward complaint-share pressure and delaying response
- Horizon-specific behavior implies model deployment should be lead-time-aware; a single global winner is not always appropriate for all planning horizons

Takeaway:

- tree models provide useful feature diagnostics, but rebuilt 02a statistical baselines remain stronger on this split and under current feature design.

Next steps:

1. add drift-aware features or regime indicators to address level shifts
2. test hybrid baseline-plus-tree correction models
3. evaluate horizon-specific model blending instead of a single winner across all horizons

10. Error Analysis (h=1): Residuals and Largest Misses

This section diagnoses one-step forecast errors for RF and XGB.

What to read from outputs:

- residual plots show direction and stability of model bias over time
- largest-miss tables identify specific months/quarters driving headline error
- summary stats (`MAE_h1` , `RMSE_h1` , mean residual) quantify average and tail behavior

Current pattern in this run:

- validation residuals are small and slightly negative (mild overprediction)
- test residuals are largely positive (systematic underprediction in later periods).

```
In [18]: # =====
# SECTION 10: ERROR ANALYSIS (H=1) FOR RF AND XGB
# =====

# Helper to select the best available h=1 predictions from prior sections.
def _pick_h1_predictions(model_prefix):
    if model_prefix == "RF":
        if "rf_val_pred" in globals() and "rf_test_pred" in globals():
            return rf_val_pred[:, 0], rf_test_pred[:, 0], "RF-GridBest"
        if "val_pred" in globals() and "test_pred" in globals():
            return val_pred[:, 0], test_pred[:, 0], "RF-Multi"
        if "rf_best" in globals() and "X_val_scaled" in globals() and "X_test_scaled" in globals():
            _v = rf_best.predict(X_val_scaled)[:, 0]
            _t = rf_best.predict(X_test_scaled)[:, 0]
            return _v, _t, "RF-GridBest (recomputed)"
        if "rf" in globals() and "X_val_scaled" in globals() and "X_test_scaled" in globals():
            _v = rf.predict(X_val_scaled)[:, 0]
            _t = rf.predict(X_test_scaled)[:, 0]
            return _v, _t, "RF-Multi (recomputed)"
        raise RuntimeError("RF predictions/models not found. Rerun Section 2 and/or 3")

    if model_prefix == "XGB":
        if "xgb_val_best" in globals() and "xgb_test_best" in globals():
```

```

        return xgb_val_best[:, 0], xgb_test_best[:, 0], "XGB-GridBest"
    if "xgb_val_pred" in globals() and "xgb_test_pred" in globals():
        return xgb_val_pred[:, 0], xgb_test_pred[:, 0], "XGB-Multi"
    if "xgb_best_models" in globals() and "X_val" in globals() and "X_test" in
        _v = predict_xgb_by_horizon(xgb_best_models, X_val, y_train_multi.columns)
        _t = predict_xgb_by_horizon(xgb_best_models, X_test, y_train_multi.columns)
        return _v, _t, "XGB-GridBest (recomputed)"
    if "xgb_models" in globals() and "X_val" in globals() and "X_test" in globals():
        _v = predict_xgb_by_horizon(xgb_models, X_val, y_train_multi.columns)
        _t = predict_xgb_by_horizon(xgb_models, X_test, y_train_multi.columns)
        return _v, _t, "XGB-Multi (recomputed)"
    raise RuntimeError("XGB predictions/models not found. Rerun Section 3 and/o

raise ValueError("model_prefix must be RF or XGB")

# Build aligned actual-vs-pred frames for val/test at h=1.
val_index = (y_val_multi.index + 1).to_timestamp(how="end")
test_index = (y_test_multi.index + 1).to_timestamp(how="end")

actual_val = y_val_multi["y_h1"].values
actual_test = y_test_multi["y_h1"].values

rf_val_h1, rf_test_h1, rf_label = _pick_h1_predictions("RF")
xgb_val_h1, xgb_test_h1, xgb_label = _pick_h1_predictions("XGB")

error_rows = []
for split_name, idx, y_true, rf_pred, xgb_pred in [
    ("Val", val_index, actual_val, rf_val_h1, xgb_val_h1),
    ("Test", test_index, actual_test, rf_test_h1, xgb_test_h1),
]:
    for model_name, pred in [(rf_label, rf_pred), (xgb_label, xgb_pred)]:
        resid = y_true - pred
        tmp = pd.DataFrame({
            "Split": split_name,
            "Quarter": idx,
            "Model": model_name,
            "Actual": y_true,
            "Pred": pred,
            "Residual": resid,
            "AbsError": np.abs(resid),
        })
        error_rows.append(tmp)

error_df = pd.concat(error_rows, ignore_index=True)

# Residual plots for h=1 (Val/Test).
fig, axes = plt.subplots(2, 1, figsize=(11, 7), sharex=False)
for ax, split_name in zip(axes, ["Val", "Test"]):
    s = error_df[error_df["Split"] == split_name].copy()
    for model_name in s["Model"].unique():
        ms = s[s["Model"] == model_name]
        ax.plot(ms["Quarter"], ms["Residual"], marker="o", label=model_name)
    ax.axhline(0.0, color="black", linewidth=1, alpha=0.7)
    ax.set_title(f"Residuals (Actual - Pred), h=1, {split_name}")
    ax.set_ylabel("Residual")

```

```

ax.legend(fontsize=8)

axes[-1].set_xlabel("Quarter")
plt.tight_layout()
plt.show()

# Largest misses by absolute error.
top_misses = (
    error_df.sort_values(["Model", "Split", "AbsError"], ascending=[True, True, False])
    .groupby(["Model", "Split"], as_index=False)
    .head(3)
    .reset_index(drop=True)
)

for c in ["Actual", "Pred", "Residual", "AbsError"]:
    top_misses[c] = top_misses[c].round(3)

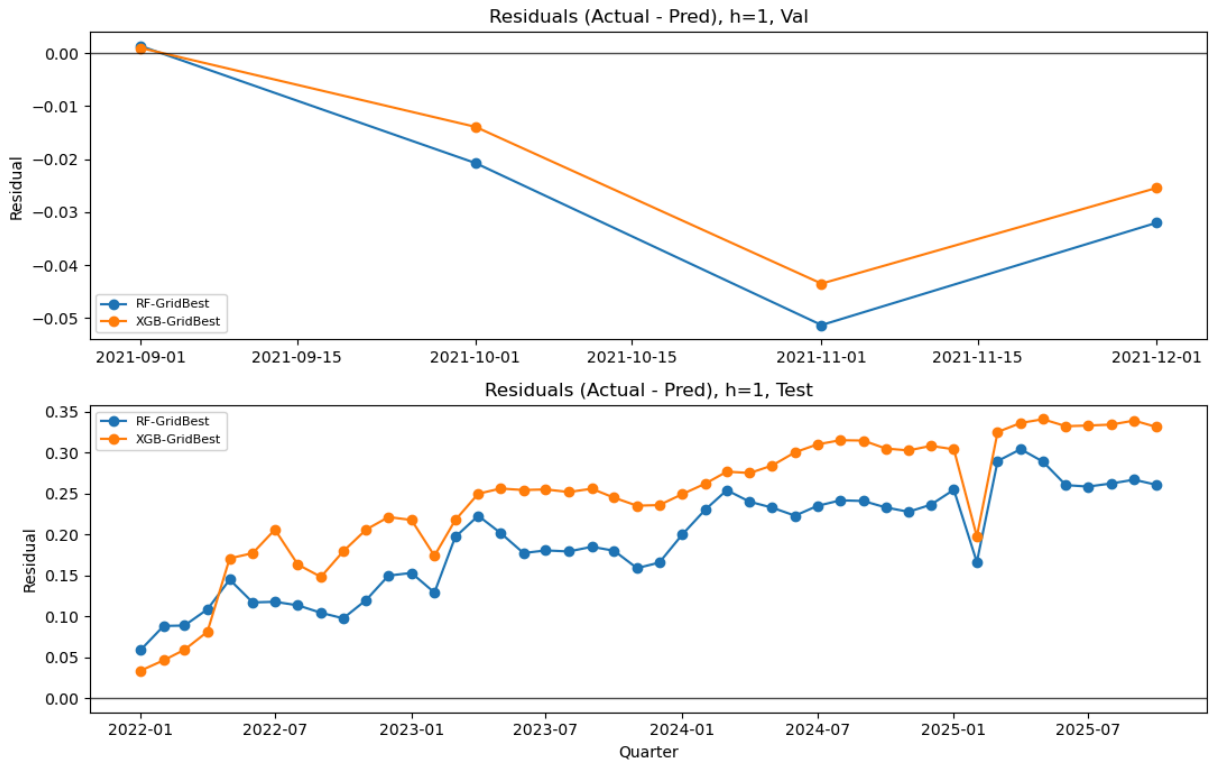
print("Top 3 largest misses per model and split (h=1):")
display(top_misses[["Model", "Split", "Quarter", "Actual", "Pred", "Residual", "AbsError"]])

# Compact summary stats for quick comparison.
error_summary = (
    error_df.groupby(["Model", "Split"], as_index=False)
    .agg(
        MAE_h1=("AbsError", "mean"),
        RMSE_h1=("Residual", lambda s: float(np.sqrt(np.mean(np.square(s))))),
        MeanResidual=("Residual", "mean"),
    )
)

for c in ["MAE_h1", "RMSE_h1", "MeanResidual"]:
    error_summary[c] = error_summary[c].round(3)

print("Residual summary (h=1):")
display(error_summary)

```



Top 3 largest misses per model and split (h=1):

	Model	Split	Quarter	Actual	Pred	Residual	AbsError
0	RF-GridBest	Test	2025-03-31 23:59:59.999999999	0.892	0.587	0.304	0.304
1	RF-GridBest	Test	2025-02-28 23:59:59.999999999	0.876	0.587	0.290	0.290
2	RF-GridBest	Test	2025-04-30 23:59:59.999999999	0.896	0.607	0.289	0.289
3	RF-GridBest	Val	2021-10-31 23:59:59.999999999	0.603	0.655	-0.051	0.051
4	RF-GridBest	Val	2021-11-30 23:59:59.999999999	0.613	0.645	-0.032	0.032
5	RF-GridBest	Val	2021-09-30 23:59:59.999999999	0.624	0.645	-0.021	0.021
6	XGB-GridBest	Test	2025-04-30 23:59:59.999999999	0.896	0.555	0.341	0.341
7	XGB-GridBest	Test	2025-08-31 23:59:59.999999999	0.899	0.560	0.339	0.339
8	XGB-GridBest	Test	2025-03-31 23:59:59.999999999	0.892	0.555	0.336	0.336
9	XGB-GridBest	Val	2021-10-31 23:59:59.999999999	0.603	0.647	-0.043	0.043
10	XGB-GridBest	Val	2021-11-30 23:59:59.999999999	0.613	0.638	-0.025	0.025
11	XGB-GridBest	Val	2021-09-30 23:59:59.999999999	0.624	0.638	-0.014	0.014

Residual summary (h=1):

	Model	Split	MAE_h1	RMSE_h1	MeanResidual
0	RF-GridBest	Test	0.192	0.202	0.192
1	RF-GridBest	Val	0.026	0.032	-0.026
2	XGB-GridBest	Test	0.244	0.257	0.244
3	XGB-GridBest	Val	0.021	0.026	-0.020